DOCKER

It is a platform that packages Applications with their dependencies to run consistently across different enviroments.

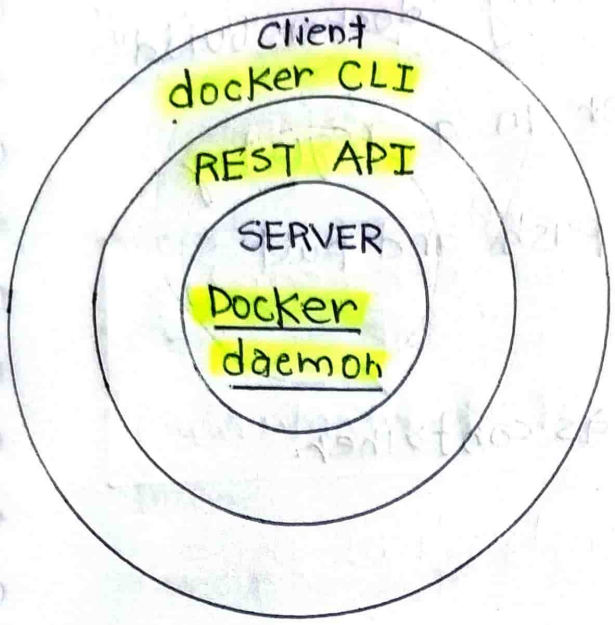
- simplifies Application Deployment
- Provides portable and Isolated enviroments.
- supports consistent Application execution.

→ Need for Docker

- **Consistency** - Ensures applications run similarly in development, testing & production.
- **Isolation** - Prevents conflicts betⁿ Application Dependencies.
- **Scalability** - Makes it easier to run multiple Application instances.
- **Portability** - Allows Applications to run across different systems and enviroments.

→ Docker Engine

It is the core component responsible for Building, running & Managing Docker containers.



↳ Main Components:

- **Docker Daemon (dockerd):**
  Background service that manages images, containers, networks & volumes.

- **Docker CLI (docker)**
  Common-line tool used to execute Docker commands.

- **REST API**
  Enables communication betⁿ the CLI and daemon and supports automation.

→ **Docker Image**

It is a ready-only template containing everything required to run an application, including code, runtime, libaries & configuration.

↳ components:

(1) **Base Image** - starting enviroment, such as Ubuntu, Alpine, Python or Node.

(2) **Application code** - File required to run the application.

(3) **Dependencies** - Required libaries, frameworks & packages.

(4) **Metadata** - configuration information, labels & other image settings.

→ Image Lifecycle -

(1) **Creation** : Build an image using "docker build".

(2) **storage** : store it locally or in a registry

(3) **Distribution** : share it using push and pull operations.

(4) **Execution** : Run the Image as container.

→ Dockerfile

It is a text file containing instruction used to build a Docker image. Its instruction help automate image creation and reuse image layers.

→ Key Instructions :

Instruction	Purpose
FROM	specifies the base image
LABEL	Adds image metadata
RUN	Executes commands during image building
COPY	Copies files into the image.
ENV	sets enviroment variables.
WORKDIR	sets the working directory.
EXPOSE	Documents the container listening port.
CMD	Defines the default startup command
VOLUME	Defines a mount point for external storage
ARG	Defines build-time variables

→ Docker Container

It is an isolated, executable instance of a Docker image that runs an application with its Dependencies.

- Provides process and filesystem isolation.
- Can be started, stopped and removed.
- Multiple containers can run from the same image.
- Containers are generally lightweight compared with virtual machines.

→ Docker Registry

It is a service that stores and distributes Docker images.

↳ some concepts:

- **Repository** - A collection of related images.
- **Tag** :- Identies an image version, such as myapp:1.0.
- **Push** - Uploads an image to a registry.
- **Pull** - Downloads an image from a registry.

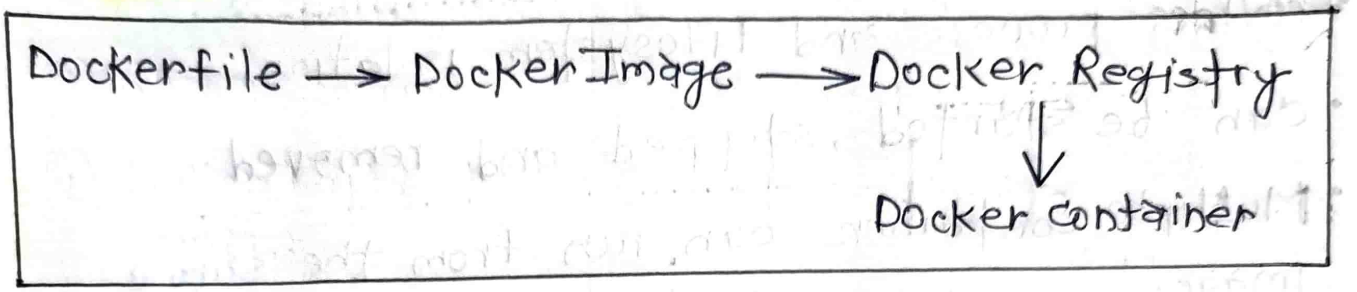
↳ Types:

- **Docker Hub** - Public and private Image repositories.
- **Private Registry** - Registry managed for an organization.
- **Cloud Registries** - Amazon ECR, Google Artifact Registry, and Azure container Registry.

→ Docker Lifecycle

It describes how an application moves from configuration to execution.

↳ Flow



- (1) Write - Define Application instructions in a Dockerfile.
- (2) Build - create an image using "docker build".
- (3) store & share - Push or pull images through a registry.
- (4) Run - create and execute a container using "docker run"

→ How Docker works

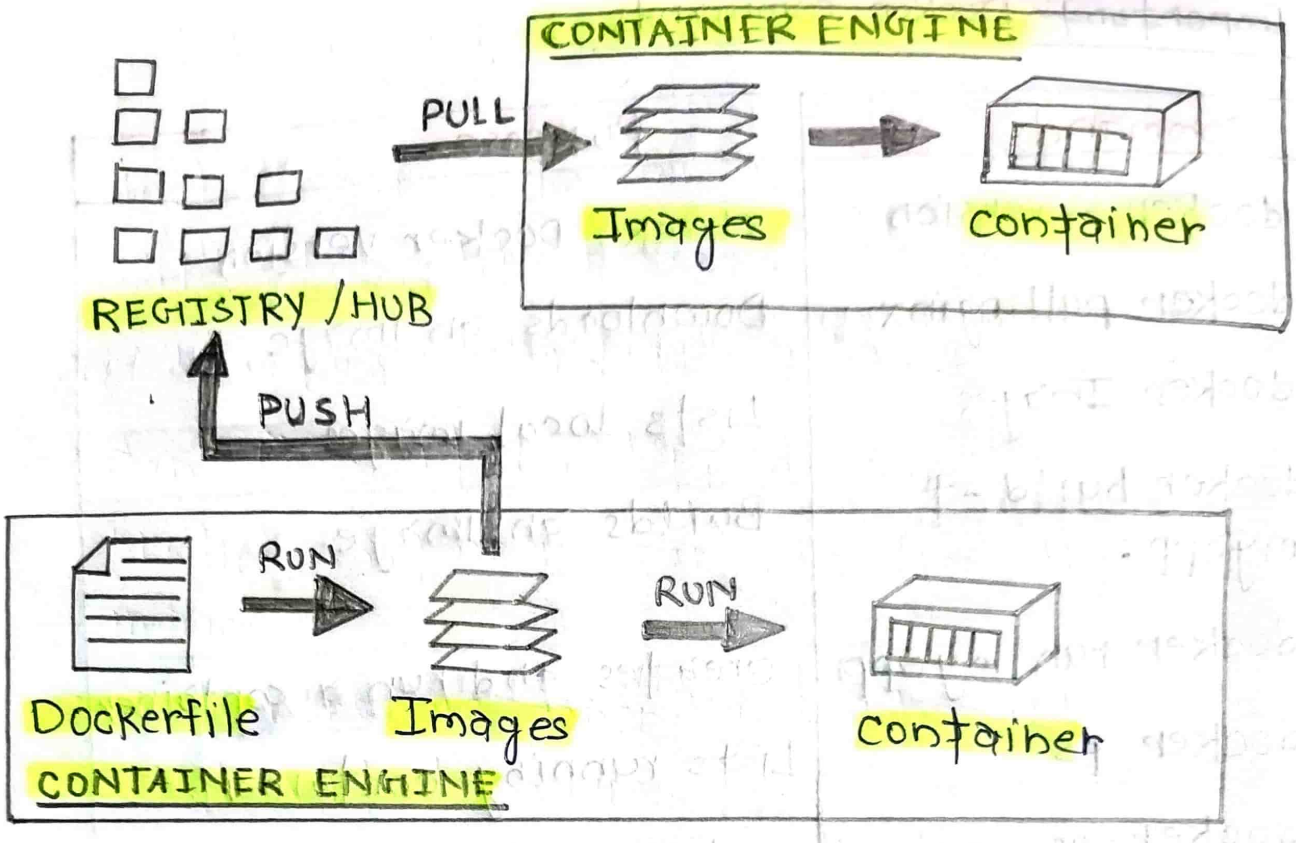
(1) Dockerfile - Defines instructions to build an image

(2) Docker Engine - Builds & manages images & containers.

(3) Docker Image - contains Application code & dependencies.

(4) Docker Registry - stores and distributes images.

(5) Docker container - Runs the Application from an image.



→ Docker Use cases

- Microservices - Runs independent services in seperate containers.
- CI/CD - Automates consistent building testing & Deployment.
- Cloud Migration - Simplifies moving application across enviroment or one service cloud provide like (aws) to another (azure).
- Web Applications - simplifies deployment and scaling
- Testing and QA - creates consistent testing enviroments.
- AI/ML - Ensures reproducible enviroments for model training & inference.
- API Development - Packages and Deploys API's reliably.

→ Important Docker commands

Command	Purpose
(1) docker --version	checks Docker version
(2) docker pull nginx	Downloads an image
(3) docker Images	Lists local images
(4) docker build -t myapp.	Builds an image
(5) docker run myapp	creates and run a container
(6) docker ps	Lists running containers.
(7) docker ps -a	Lists all containers
(8) docker stop CONTAINER_ID	stops a container
(9) docker rm CONTAINER-ID	Removes a container
(10) docker push USER/myapp:1.0	Pushes an image to a registry

→ Docker setup

Docker installation depends on the operating system.

- Windows - Install Docker Desktop & enable WSL 2 for Linux container support.

*Add your Id/login in Docker hub for registry like Reporistory in Github

Verify Docker Installation

Use these commands for check installing and working.

```
docker --version
docker info
docker run hello-world
```

It is like "Hello world" for Docker for checking

How "docker run Hello-world" works:

(1) Docker CLI sends a request to the Docker daemon.

(2) Docker checks whether the Image exists locally

(3) If unavailable, Docker downloads it from docker hub.

(4) Docker creates & runs a container from the image.

(5) The container display "Hello from Docker!".

→ Pulling an Image from Github container Registry (GHCR)

- GHCR is a Github's container registry for storing & distributing Docker image.

Step: 1 - creates a Personal Access Token (PAT)

- open github settings.
- Navigate to Developer settings.
- creates a personal access token with the required package permissions, such as "read:packages" for Pulling images.

→ **Step:2** - Log in to GHCR

```
export CR_PAT=your_personal_access_token

echo "$CR_PAT | docker login ghcr.io \
-u YOUR_GITHUB_USERNAME --password-stdin
```

↳ **Step:3** - Pull an Image

```
docker pull ghcr.io/OWNER/IMAGE:TAG
```

↳ **Step:4** - Run the Image

```
docker run -d -p 8080:80 ghcr.io/OWNER/IMAGE:TAG
```

- -d = Runs the container in the Background.
- -p 8080:80 = Maps host port 8080 to container port 80.
- OWNER/IMAGE:TAG = Replace with the actual image owner, name & tag.

Note - The Image must exist, & your account must have permission to Access it.

→ Dockerizing a Flask Application

↳ **Dockerizing** - It means packaging an application and its dependencies into a Docker image so it can run consistently across environments.

↳ **Step: 1** - Create the Application

File - app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Dockerized
    Flask App!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

* The Application creates a Flask web server that returns a message when someone visits the home route.

↳ **Step: 2** - Create "requirements.txt"

```
flask==3.0.0
```

* This file specifies the python dependency required by the application.

Step : 3 - Creates a Dockerfile

File : Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python","app.py"]
```

- Explanation :-

* FROM - selects the python base image.
* WORKDIR - sets the working directory.
* COPY - copies Application file into the image.
* RUN - Install the required dependencies.
* EXPOSE - Documents the application's listening port.
* CMD - Defines the command that starts the application.

Step : 4 - Build the docker image.

```
docker build -t flask-app .
```

* build - creates the image.
* -t flask-app - Assigns the image name.
* . - Uses the current directory as the build context.

→ **step : 5-** Run the container

```
docker run -d -p 5000:5000 --name
my-running-flask flask-app
```

* -d = Runs in the background
* -p 5000:5000 = Maps host port 5000 to container port 5000
* --name = Assigns a container name.
* flask-app - specifies the image

• **Open** - http://localhost:5000 in your browser to access the Application.

Note - The "requirements.txt" file should follow these structure for hustle free operations.

(1.) base image
(2) workdir
(3) copy
(4) run
(5) port
(6) command

```
#base image
FROM python:3.9
# workdir
WORKDIR/app
# copy
COPY ./app
# run
RUN pip install -r requirements.txt
#port
EXPOSE 5000
#command
CMD ["python", "/app.py"]
```

→ Pushing an Image to Docker Hub

Pushing an image to Docker Hub makes it Available for sharing and deployment on other machines.

↳ Step: 1 - Log in

```
docker login
```

It Authenticate using our Docker hub credentials for an access token.

↳ Step: 2 - Tag the image

```
docker tag flask-app YOUR_USERNAME/flaskapp:V1.0
```

The standard image naming format is:

USERNAME/REPOSITORY:TAG

Here, V1.0 identifies the image version.

↳ Step: 3 - Push the image

```
docker push YOUR_USERNAME/flask-app:V1.0
```

Docker uploads the tagged image to our Docker hub repository.

↳ step: 4 - Pull and run the image on another machine.

```
docker pull YOUR_USERNAME/flask-app:V1.0
docker ryn -d -p 5000:5000 YOUR_USERNAME/
flask-app:V1.0
```

→ Some More / Additional Docker topics

- **Docker Volumes** - store data persistently even when container are removed.
- **Docker Networking** - Enables communication between container.
- **Enviroment Variables** - configure applications without changing source code.
- **.dockerignore** - Exclude unnecessary files from the build context.
- **Docker Compose** - Manages multiple containers using a YAML file.
- **Image Layers** - Help Docker reuse cached build steps.
- **Health Checks** - check whether an Application is functioning correctly.
- **Restart Policies** - Automatically restart containers under specific condns
- **Docker security** - Use trusted images & Avoid storing secret in image.
- **Docker Debugging** - Use "docker logs" to Identify Application errors.
- **Docker in MLOps** - Packages ML models & dependencies for consistent Deployment.

- **Docker in CI/CD** - Automates application building, testing & Deployment.